

Parte 8

Engenharia de Performance

CI1164 - Introdução à Computação Científica
Profs. Armando Delgado e Guilherme Derenievicz
Departamento de Informática - UFPR

Porque medir performance?

- Determinar o computador mais adequado à aplicação:
 - Gaming PC, Laptop, HPC?
 - Cluster ou Servidor? Qual CPU? Qual GPU?
- Avaliar o impacto de novas implementações / otimizações:
 - Resultados precisam ser comparáveis entre versões diferentes
 - Externa e internamente
- Determinar a capacidade dos componentes do computador:
 - Transferência de dados
 - Capacidade de E/S e de computação
 - Largura de banda de rede

O que é uma boa métrica?

- Performance = **TRABALHO / TEMPO**
- Definindo **TRABALHO**
 - **MFlop/s**: **Milhões de Operações de Ponto Flutuante** por Segundo
 - Aplicações científicas, computação numérica.
 - **MIPS**: **Milhões de Instruções** por Segundo
 - Banco de dados, servidores web
- Como calcular **TRABALHO** (p.ex. **Flop**)?
 - Contar manualmente (no código)
 - Usar eventos de CPU → **LIKWID**

Torturando números

- “Meu código executa 2000 MFlop/s em um processador de 2GHz!”

```
for (i=0; i < n; i++){  
    a[i] = 3.0 * c0 + c1*c2 + c3*c4*a[i] - 1.0*a[i];  
}
```

➔ **#FLOP = 8*n**

- Mesmo tempo de execução, mas 1/4 da taxa de MFlop/s

```
d0 = 3.0*c0 + c1*c2;  
d1 = c3*c4 - 1.0;  
for (i=0; i < n; i++){  
    a[i] = d0 + d1*a[i];  
}
```

➔ **#FLOP = 2 * n + 5**

Escolhendo a região de **TRABALHO**

- **Iterações:** número de iterações em um laço
 - $\text{TRABALHO} = n$
 - Métrica: **Iterações / s**
- **Atualizações de Malhas / Células / Partículas:** usado em simulações
 - $\text{TRABALHO} = \text{número de células a serem computadas}$
 - Métrica: **células atualizadas / s**
- **Tempo para a solução:** problemas bem definidos
 - $\text{TRABALHO} = \text{"1" problema bem definido}$
 - Métrica: **1 / s**

TEMPO

- **Métrica mais simples:** $1 / \text{TEMPO}$
 - Mede tempo para a solução
 - Cuidado ao especificar o “problema” a ser resolvido
 - Melhor métrica, mas requer cuidado nas definições.
- **Problema:** como medir TEMPO?
 - `time (BASH) x /usr/bin/time (POSIX)`
 - > `/usr/bin/time ./cgSolver ...`
 - `15.30 user 0.02 system 0:15.33 elapsed 99%CPU`
 - USER CPU, SYSTEM CPU, Elapsed time, CPU ratio

TEMPO

- Execução de **TEMPO DE CPU** – depende do número de *threads*,
- *Elapsed time* (*wall-time*) é o tempo para a solução
 - Sempre use recursos **EXCLUSIVOS / DEDICADOS** (!)
- Medindo *wall-time*

→ Use **gettimeofday()**

```
#include <sys/time.h>
```

```
double timestamp(void){
```

```
    struct timeval tp;
```

```
    gettimeofday(&tp, NULL);
```

```
    return((double)(tp.tv_sec+tp.tv_usec/1000000.0)); }
```

→ **WALL-TIME**: diferença entre dois *timestamps*!

Profiling: Onde se está gastando mais tempo?

- Como saber onde o código gasta mais tempo?
 - Implemente o código de forma que o tempo de cada função pode ser determinado
 - Usar ferramentas de *profiling*: Intel Amplifier, Oprofile, Code analyst
 - *Profiling* influencia performance: use para obter informação **qualitativa**

Profiling com gprof

- Mede tempo e uso de funções no programa
- Compilar com opção para *profiling*
 - > gcc -pg <source.c> -o program
 - > ./program
- Depois de execução, arquivo **gmon.out** é criado
 - > gprof ./program gmon.out
- Desabilite “funções *inline*” para *profiling*
 - Mas aí o código não será o mesmo ...
- *Profiling* pode reduzir a performance de seu código

Medindo Performance de Hardware

- Quando um trecho de tensão (*hot spot*) é identificado
 - Determinar o uso de hardware
- **Contadores de Performance** permitem a monitoração de eventos do processador
 - Quantidade e tipo de instruções executadas
 - Transferência de dados entre os diferentes níveis de memória
 - Tempo de execução
 - Consumo de energia
- Muitos eventos (+500), mas poucos contadores (4-10)

LIKWID (**L**ike **I** **K**new **W**hat **I**'m **D**oing)

- Monitora contadores de performance do sistema
 - suporta medida de múltiplas regiões no programa
 - Métricas pré-configuradas:
 - BRANCH, CACHE, ICACHE, CLOCK, FLOPS_DP, FLOPS_AVX, MEM, ENERGY, ...
- Permite que a alocação (afinidade) de um programa com um ou mais *cores*
- Exibe a hierarquia de memória e *cores*
- Permite a execução de testes de referência (*benchmarks*)

LIKWID

- Conjunto de ferramentas:
 - **likwid-topology**: exibe a arquitetura do processador e sua hierarquia de cache
 - **likwid-bench**: executa micro-benchmarks no sistema para testar os limites do processador.
 - **likwid-perfctr**: monitora contadores de performance do Sistema (Socket, Processador, Core).
 - **likwid-perfscope**: exibe graficamente a evolução dos contadores de performance

likwid-topology

- `likwid-topology -c -g`

```
*****
Graphical Topology
*****
Socket 0:
+-----+
| +-----+ +-----+ +-----+ +-----+ |
| | 0 4 | | 1 5 | | 2 6 | | 3 7 | |
| +-----+ +-----+ +-----+ +-----+ |
| | 32 kB | | 32 kB | | 32 kB | | 32 kB | |
| +-----+ +-----+ +-----+ +-----+ |
| +-----+ +-----+ +-----+ +-----+ |
| | 256 kB | | 256 kB | | 256 kB | | 256 kB | |
| +-----+ +-----+ +-----+ +-----+ |
| +-----+ +-----+ +-----+ +-----+ |
| |                                     8 MB | |
| +-----+ +-----+ +-----+ +-----+ |
+-----+
```

likwid-perfctr

- **likwid-perfctr -e**: lista os contadores de performance e eventos disponíveis no sistema
- **likwid-perfctr -a**: lista os grupos de performance disponíveis no sistema
 - Os grupos combinam dados de diversos contadores
- Contadores e grupos variam entre diferentes processadores.

likwid-perfctr

- Duas formas de utilizar:

- Executando sobre todo o programa

- `likwid-perfctr -C <core> -g <group> <program-to-monitor>`

- Instrumentando o código fonte com uma API, isolando apenas o(s) trecho(s) a serem monitorados

- `likwid-perfctr -C <core> -g <group> -m <program-to-monitor>`

likwid-perfctr

- Marker API:

```
#include <likwid.h>
int main(int argc, char *argv[]) {
    ...
    LIKWID_MARKER_INIT;
    ...
    LIKWID_MARKER_START("marker-name"); // Nome do marcador SEM ESPAÇOS
    ... <o código que se quer medir vai aqui>
    LIKWID_MARKER_STOP("marker-name");
    ...
    LIKWID_MARKER_CLOSE;
}
```

→ Compilação:

- ▶ **gcc -DLIKWID_PERFMON** -I<PATH_TO_LIKWID>/include -L<PATH_TO_LIKWID>/lib -llikwid
- ▶ <PATH_TO_LIKWID> → **/usr/local** (instalação padrão)
 - ▷ **/home/soft/likwid** no (DINF/UFPR)

LIKWID

- Projeto
 - <https://github.com/RRZE-HPC/likwid>
 - <https://github.com/RRZE-HPC/likwid/wiki>
- Em casa:
 - Carregar módulo **msr** do kernel
 - `modprobe msr`
 - `/etc/modules`
- Variáveis de ambiente do shell:
 - `export PATH=<PATH_TO_LIKWID>/bin:<PATH_TO_LIKWID>/sbin:$PATH`
 - `export LD_LIBRARY_PATH=<PATH_TO_LIKWID>/lib:$LD_LIBRARY_PATH`

Pontos importantes em otimização de performance

- Eu compreendo o comportamento da performance de meu código?
 - A performance combina com um modelo que eu fiz?
- Qual a performance ótima para meu código em uma certa máquina?
 - Computação de Alta Performance (HPC) == Computação no limite/gargalo
- Posso mudar meu código para que a “performance ótima” fique melhor?
 - Contornar / amenizar o impacto do gargalo

Pontos importantes em otimização de performance

- Meu modelo não funciona – o que está errado?
 - Este é o melhor caso, pois se aprende algo novo
 - Monitoramento de Performance / *micro-benchmarking* podem ajudar a clarear a situação
- Use seu cérebro! Ferramentas podem ajudar, mas é você que deve raciocinar.

Fatores de impacto em performance

- Um código pode ter sua performance influenciada por diversos fatores:
 - **CPU:** relógio, # cores, tamanho de cache, *threads* simultâneas (SMT)
 - **Memória:** interface, tipo, velocidade, tamanho, fabricante, barramentos, placa mãe, sistemas de E/S
 - **Compilador:** Intel, gcc, ibm, clang, microsoft, versão
 - **Bibliotecas:** atlas, mkl, libc, fftw, versão, compilador
 - **SO:** distribuição, parâmetros
 - **BIOS:** várias definições

Fatores de impacto em performance

- Para permitir comparação/reprodução, todos os fatores críticos devem ser reportados
- Sempre fazer análise de estabilidade e sensibilidade
- Estatística: variabilidade entre execuções.

Medidas de Performance: boas práticas

- Preparação
 - Medida de tempo confiável (granularidade)
 - Documentar geração de código (flags e versão do compilador)
 - Documentar estado do sistema (*clock*, modo turbo, memória, caches)
 - Considere criar *scripts* para automatizar os testes

Medidas de Performance: boas práticas

- Ao executar:
 - Tenha um sistema **EXCLUSIVO**.
 - Fixar a frequência do processador (performance x economia de energia) !
 - Controle afinidade / topologia: onde exatamente meu código está executando?
 - O resultado é determinístico e reproduzível?
 - Desempenho na média? Melhor? Pior?
 - Variáveis básicas: qtd. *threads*, afinidade, e tamanho da entrada
 - Checar se os resultados são razoáveis:
 - ▶ MEM: 1.0e-308 GB/s ?????

Medidas de Performance: boas práticas

- Pós-processamento
 - Documentar o que foi feito
 - Tentar entender e explicar o resultado
 - “rubber duck method”
 - Planejar variações para mais informações
 - Muitas coisas ficam mais claras em um gráfico
 - `gnuplot`, `xmgrace`
- Existe um modelo simples com o qual se pode explicar as variações e níveis de performance?

Benchmarks: Modelos de Referência

- **Aplicações Reais**: código efetivamente usado para resolver problemas. Pode tomar horas ou dias e portabilidade é um problema
- **Aplicações Modificadas**: concentra na parte mais importante (resolução, aritmética) e remove outras partes (E/S, pós-processamento). Tempo de execução reduzido quando se define um número fixo de iterações.
- **Kernels**: extrai pequenas partes do código, eg. *solver* (LINPACK), ou processos atrasados (*stream*, MVM, *Multigrid smoothing*). Portável e fácil de otimizar. Usado por vendedores.
- **Benchmarks**: sintético (diferentes operações típicas de HW), ou jogos (eg. ordenação) para propósitos educacionais e de teste.

Benchmarks: padrões em HPC

- LINPACK → Top500 / Máxima Performance
- STREAM → Máxima banda de memória
- HPCG → Resolve Sistemas Lineares por Gradiente Conjugado
- SPEC → Padrão da indústria (não apenas HPC)

LINPACK

- Resolve grandes Sistemas de Equações Lineares
 - Fatoração LU de matriz A
 - Complexidade: $\frac{2}{3}N^3 + O(N^2)$
 - Implementações altamente paralelas
 - Quase atinge performance teórica de máquina
 - <http://www.netlib.org/benchmark/hpl/algorithm.html>

STREAM

- Executa 4 operações “*streaming*”:
 - COPY: $A[] = C[]$
 - SCALE: $A[] = s * C[]$
 - ADD: $A[] = B[] + C[]$
 - TRIAD: $A[] = B[] + s * C[]$
- Resultados em MB/s
- Muito usado em exemplos desta disciplina
- **LIKWID** implementa estes *benchmarks*

LIKWID: `likwid-bench`

- `likwid-bench -i <iterações> -t <benchmark> -w <workgroup>`
 - ➔ **Iterações:** Número de iterações a executar o benchmark.
 - ➔ **Benchmark:**
 - `likwid-bench -a` → lista de *benchmarks*
 - `likwid-bench -l <benchmark>` → propriedades de um *benchmark*
 - ➔ **Workgroup:** sockets, threads, tamanho
 - ➔ **Exemplos:**
 - `likwid-bench -i 100 -t triad -w S0:1000MB:1`
 - `likwid-bench -i 100 -t triad_avx -w S0:1000MB:1`
 - `man likwid-bench` → documentação

HPCG

- **H**igh **P**erformance **C**onjugate **G**radient
- Gradiente Conjugado com pré-condicionador simétrico Gauss-Seidel
- Sistemas lineares sintéticos, esparsos e 3D
- Alta correlação com **STREAM** por ser limitado em memória (*memory bound*)

SPEC

- **S**tandard **P**erformance **E**valuation **C**orporation
 - www.spec.org
- Diferentes categorias de *benchmarks*
 - CPU
 - Computer graphics / workstation / desktop
 - MPI / OpenMP
 - Java Client / Server
 - E-mail server
- Resultados reportados em *wall-clock time* e comparado com sistemas de referência

O que NÃO fazer?

- “Fooling the masses with performance results: Old classics and some new ideas”
 - G. Wellein & G. Hager
 - <https://blogs.fau.de/hager/archives/5260>

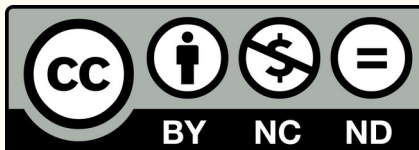
Referências

- Daniel Weingaertner; notas de aula da disciplina **Introdução à Computação Científica** (UFPR/DINF)
- G. Hager, G. Wellein; **Introduction to High Performance Computing for Scientists and Engineers**. CRC Press, 2011.

Créditos

Este documento é de autoria do Prof. Armando Luiz N. Delgado (UFPR/DINF), para uso na disciplina Introdução à Computação Científica (CI1164), a partir de material de autoria do Prof. Daniel Weingaertner (UFPR/DINF).

Compartilhe este documento de acordo com a licença abaixo



Este documento está licenciado com uma Licença Creative Commons **Atribuição-NãoComercial-SemDerivações** 4.0 Internacional.

<https://creativecommons.org/licenses/by-nc-sa/4.0/>